

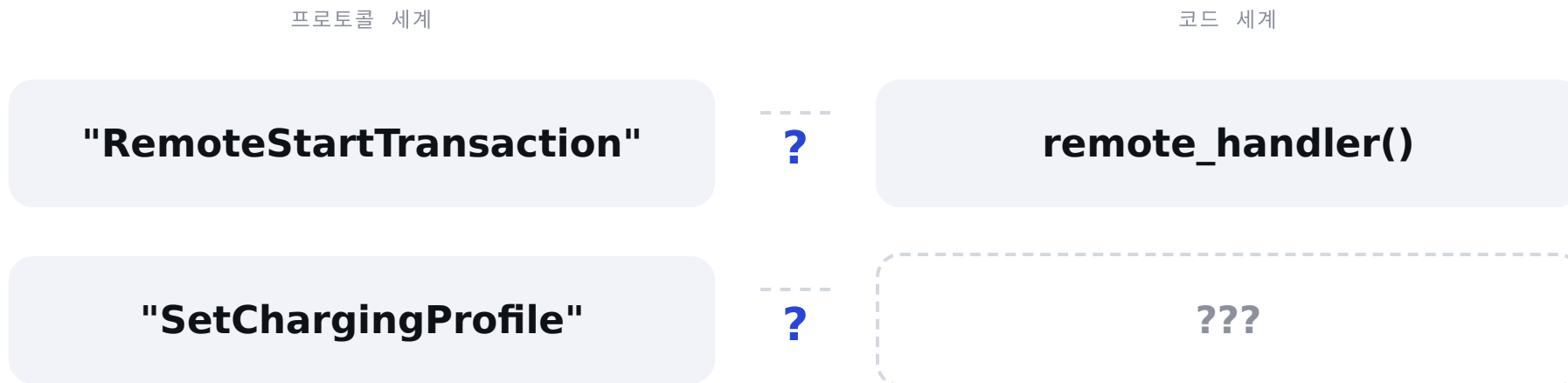
동적 디스패치가 감춘 핸들러를, 증거로 되찾다

F2-A — 프로토콜 액션을 소스 코드의 핸들러로 잇는 정적 분석 단계.
추측하지 않고, **증거로 판정**한다.

KB > **F2-A** > 필드 바인딩 > 데이터 흐름 > 싱크 > 취약점 후보

프로토콜의 의미와 코드는 꿈겨 있다

액션은 명세에, 이를 처리하는 코드는 간접 호출 너머에 있다.



연결은 제어 흐름이 아니라 데이터 속에 숨어 있다

콜그래프에는 핸들러로 가는 간선이 없다

함수 포인터·디스패치 테이블이 **caller→handler** 간선을 지운다. 연결은 데이터에만 남는다.

제어 흐름 · 콜그래프

dispatch(frame)



(*table[i].fn)()



× 정적 간선 없음

remote_handler()

데이터 · 등록 지점

table[k] = { ACTION, remote_handler }

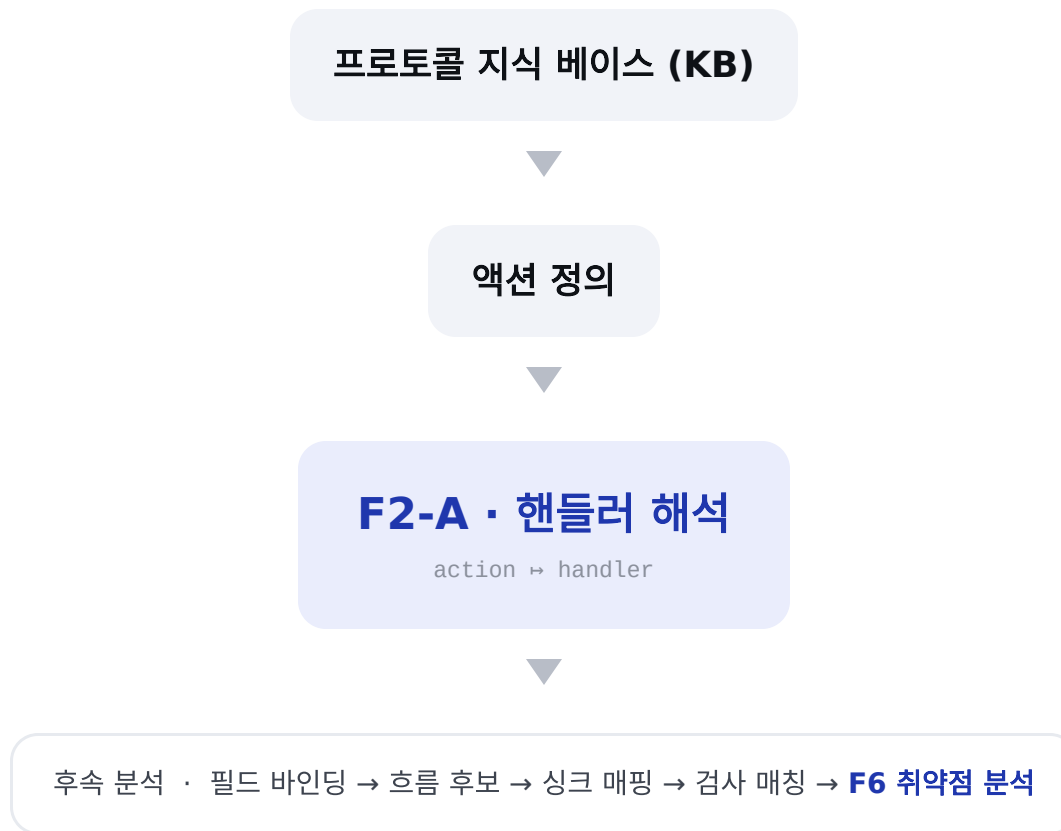


여기에 간선이 있다

remote_handler()

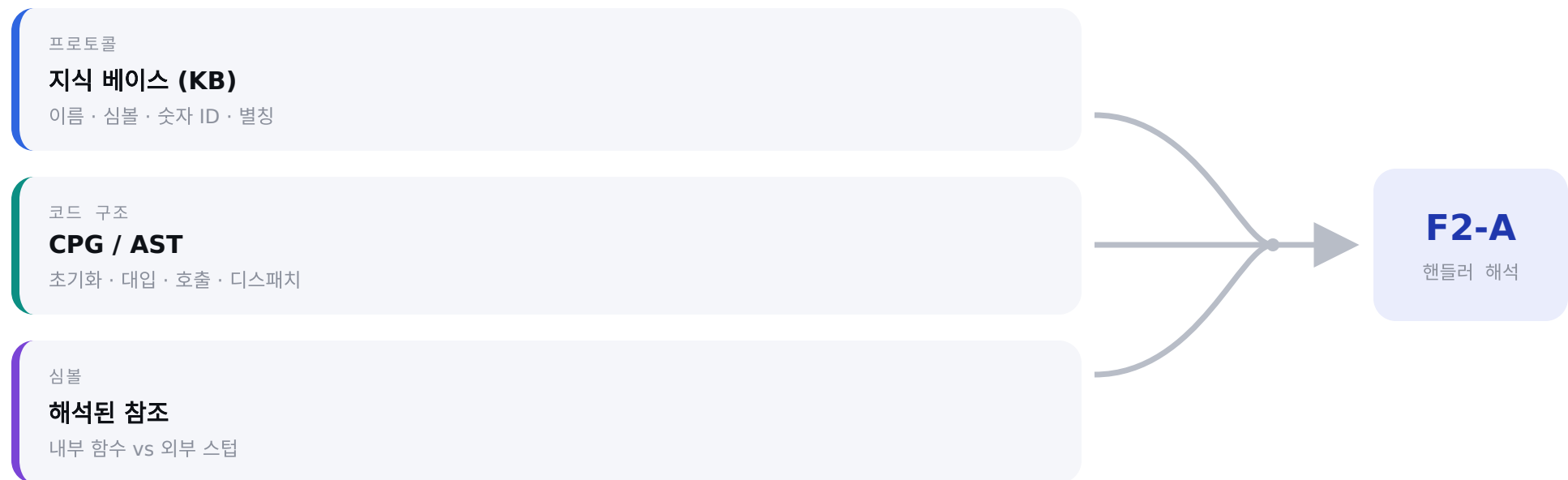
F2-A가 풀려야 후속 분석이 시작된다

액션→핸들러 앵커를 만드는 관문 단계. 이후 모든 프로토콜 인지 분석이 이걸 소비한다.



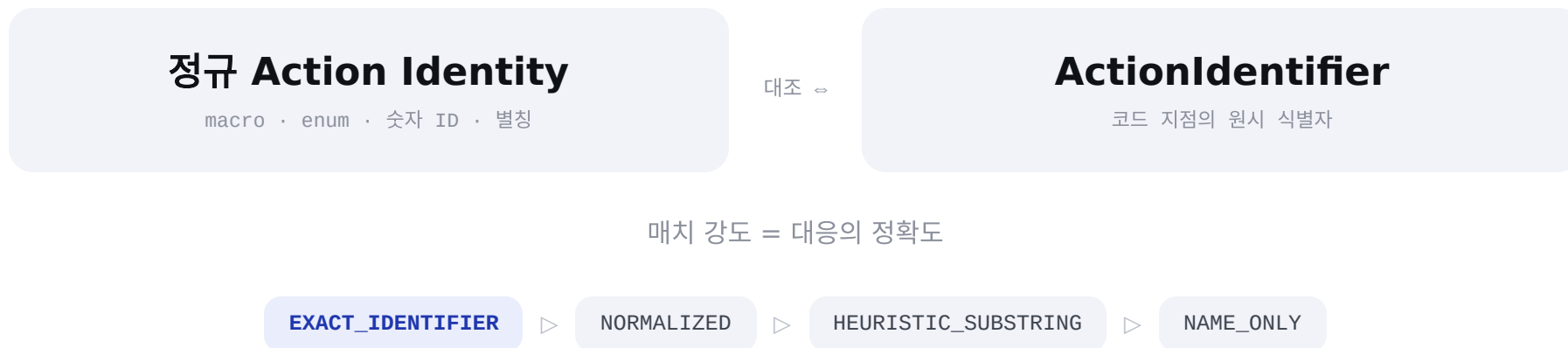
프로토콜 진실과 코드 구조가 만나는 곳

독립된 세 도메인 — 프로토콜·구조·심볼 — 이 하나의 해석기로 모인다.



하나의 액션, 여러 표기를 하나로 정규화한다

코드의 원시 식별자를 정규 **ActionIdentifier**와 대조한다. 매치 강도는 여기서 나온다.



핸들러 등록은 형태가 제각각이다

의미는 하나(action $\mapsto$ handler), 문법은 여러 가지. 그래서 인식기도 여럿이어야 한다.

집합 초기화 · **aggregate**

```
{ ACTION, fn }
```

지정 초기화 · **designated**

```
{ .action = ACTION, .fn = fn }
```

인덱스 대입 · **indexed**

```
handlers[ACTION] = fn
```

상관 저장 · **correlated**

```
t[k].action = ACTION; t[k].fn = fn
```

등록 함수 · **registrar**

```
register(ACTION, fn)  $\rightarrow$  콜리가 저장
```

위임 등록 · **delegated**

```
register(...)  $\rightarrow$  store_handler(...)  $\rightarrow$  t[.] = fn
```

추출기는 인식할 뿐, 판정하지 않는다

여러 인식기가 하나의 증거 스트림으로 모이고, 그 뒤는 모두 별개의 계층이다.

aggregate designated indexed correlated registrar delegated dispatch name

하나의 증거 스트림으로 수렴 ▼



어떤 계층도 자기 위 계층의 결정을 대신하지 않는다

후보를 가르는 건 종류가 아니라 매치 강도다

디스패치와 등록의 사전확률은 대등하다 — 어떤 종류도 해석에서 배제되지 않는다.

등록 증거 `explicit id ↔ callback`

`aggregate` → `REGISTRATION_INIT`

`designated` → `REGISTRATION_INIT`

`indexed/field` → `REGISTRATION_ASSIGN`

`registrar` → `REGISTRAR_CALL`

`delegated` → `REGISTRAR_CALL`

디스패치 · 이름 증거 `invocation / lexical`

`dispatch` → `ENUM_CASE`

`dispatch` → `STRING_DISPATCH`

`name` → `NAME_MATCH`

디스패치는 **높은 사전확률·정확 매치 가능**, 이름은 약한 폴백

제한 요인 = **정확 식별자 매치의 부재** (weak-only cap) — 종류와 무관

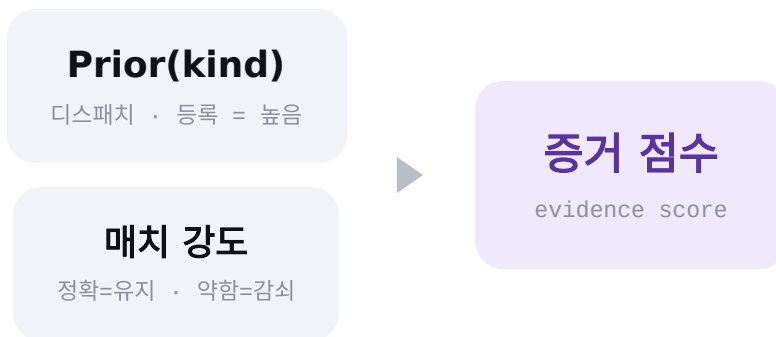
모든 추출기는 하나의 증거로 말한다

출처와 파일·라인 추적이 담긴, 타입화된 증거 레코드.

EVIDENCE · 레코드	
action id	대조된 식별자 (+ 매치 강도)
callback	해석된 내부 함수
kind	어떤 방식이 만들었는가
provenance	사이트/그룹 키 (중복 제거·그룹화)
confidence	$\text{Prior}(\text{kind}) \times \text{매치 강도}$
records[]	파일·라인 추적 (감사 가능)

증거 점수 = 사전확률 × 매치 강도

개별 증거 하나의 점수. 아직 후보를 합치지 않는다.



개별 점수일 뿐 — 후보 병합은 다음 단계

중복을 접고, 독립 근거만 합친다

사이트 내 MAX로 중복 제거 → 사이트 간 noisy-OR로 확증 → 상한·감점.

한 후보의 여러 증거 점수



1단계 · 접기 — 사이트 내 MAX

같은 사이트 = 새 정보 아님



2단계 · 결합 — 사이트 간 noisy-OR

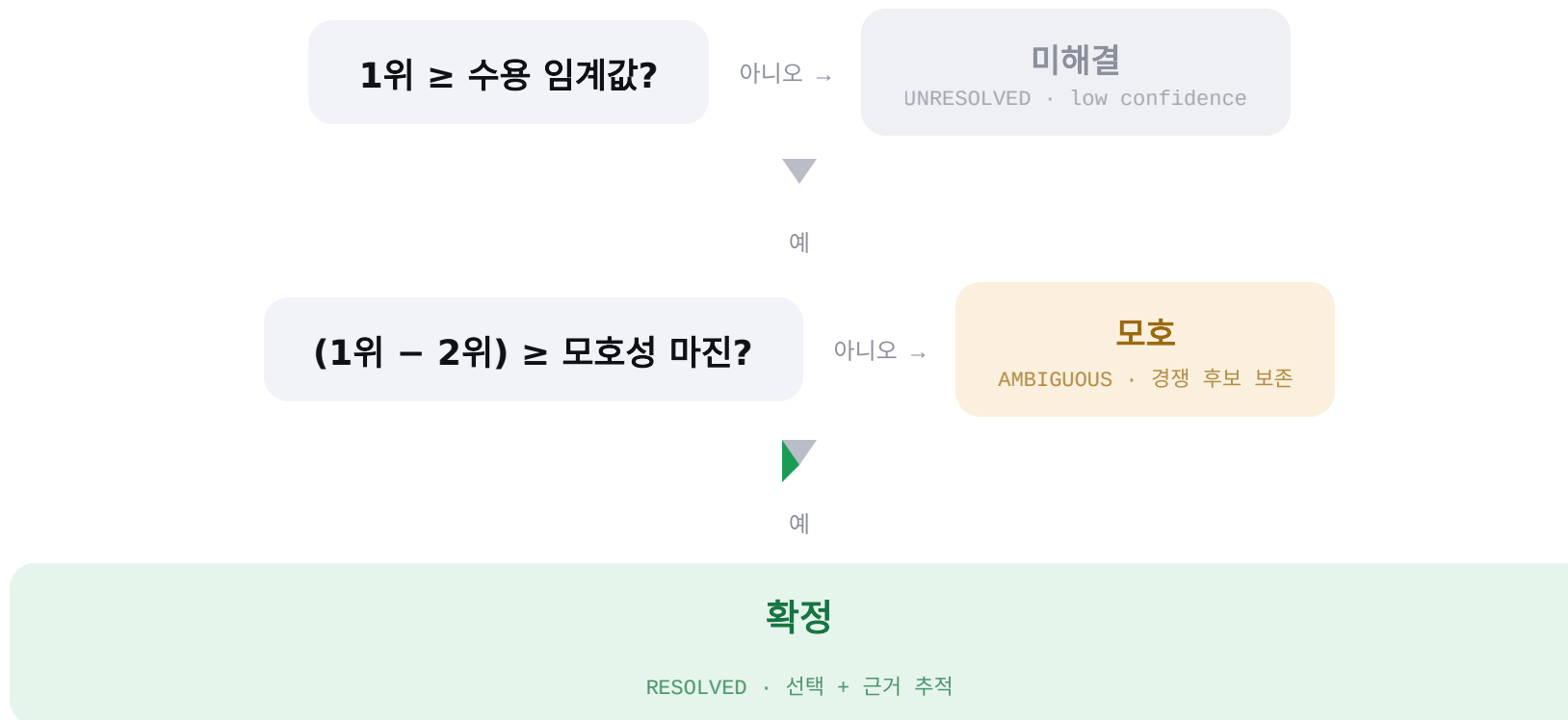
독립 확증 → 신뢰 ↑



Weak-only Cap · Global Cap · Conflict Penalty

확신만으로 확정·보류·미해결을 가른다

수용 임계값(Acceptance Floor), 그다음 모호성 마진(Ambiguity Margin) — 구조 기반 게이트는 없다.



구조 종류 게이트 없음 — 약한 매치 후보는 감점될 뿐, 배제되지 않는다

미해결은 실패가 아니라 ‘막힌 지점’이다

모든 미해결에는 구조화된, 실행 가능한 사유가 붙는다.

증거 없음	NO_EVIDENCE				
임계값 미달	LOW_CONFIDENCE				
분석 한계	REGISTRAR_STORE_NOT_REACHED	EXTERNAL_DEFINITION	MISSING_POINTSTO	REGISTRATION_OUT_OF_TU	
미지원 패턴	REGISTRAR_SEARCH_THEN_WRITE	DYNAMIC_ACTION_ID			

‘신호 없음’과 ‘의도적으로 따라가지 않는 신호’를 구분 — 버그가 아니라 **범위 선언**

이 사례에선 디스패치 증거가 이긴다

두 추출기가 발화 — 더 강한 식별자 매치가 확정을 가져간다.

```
static Reg t[] = { { ACTION_DATA_TRANSFER, foo } }; // 등록
switch (f->action){ case ACTION_DATA_TRANSFER: bar(); } // 디스패치
```

증거 · EVIDENCE

A · REGISTRATION_INIT → foo · id=ACTION_DATA_TRANSFER (부분 문자열)
B · ENUM_CASE → bar · id=ACTION_DATA_TRANSFER (정규화 일치)

bar · 0.72

vs

foo · 0.56



확정 → bar

switch 제거 시 → 증거 A만 → foo 단독 확정 (≈0.56, 임계값 턱걸이)

미해결도 하나의 정식 결과다

액션별 권위 있는 결과 집합 — 모든 후보가 근거로 연결된다.

handler_resolutions

권위 있는 결과 · 액션당 하나

status · RESOLVED/AMBIGUOUS/UNRESOLVED

chosen · 확정 시에만

candidates · 증거+추적 포함

conflict · 경쟁+마진

unresolved · 사유+시도

handler_maps

확정 전용 뷰 · 하위 호환

limitations

전역 범위 한계

확정된 핸들러가 모든 후속 분석의 기준점

필드 바인딩·소스·싱크·검사가 전부 핸들러에서 확장된다.



구조로 결정되는 범위까지가 기준선

정적으로 판정 가능한 등록·디스패치 형태 + 명시적 모호성 처리.

- ✓ 집합 초기화
- ✓ 인덱스 · 상관 필드 저장
- ✓ 위임 등록 (깊이 제한)
- ✓ enum/switch 디스패치 인식
- ✓ 모호성 탐지 · 보류
- ✓ 지정 초기화
- ✓ 등록 함수 추적 (저장 도달)
- ✓ 심볼릭 리시버 상관 (공유 슬롯)
- ✓ 이름 매치 폴백
- ✓ 구조화된 진단

값 추론이 필요한 패턴은 범위 밖으로 둔다

루프·별칭·프로시저 간 사례는 탐지·명명 후 보류한다.

제외 패턴

search-then-write 등록 함수

런타임 슬롯 선택

별칭→슬롯 바인딩

프로시저 간 슬롯 재구성

일반 동적 액션 ID

필요 역량

루프 + 슬어 추론

심볼릭 인덱스 해석

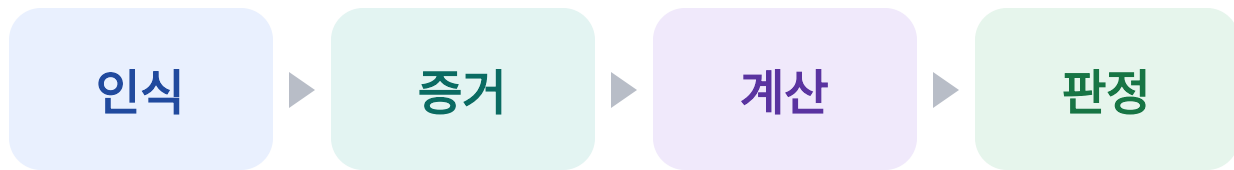
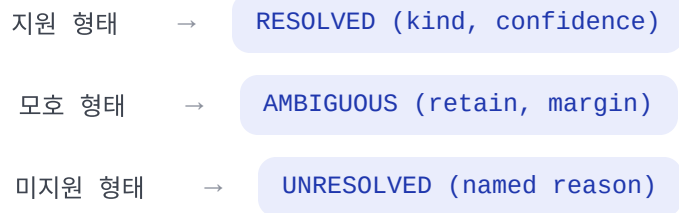
별칭 / points-to 분석

프로시저 간 값 흐름

DFG 기반 에스컬레이션

인식과 판정의 분리, 그것이 기여다

테스트가 동작을 고정하고, 백본이 확장을 가능케 한다.



“인식과 판정을 증거 추상으로 분리함으로써, F2-A는 보수적이고 · 감사 가능하며 · 자연스럽게 확장된다.”

질의응답용 참고 자료

본 서사에서 뺀 구체 파라미터와 메커니즘.

A1 · 파라미터

종류별 사전확률, 매치 강도 배수, weak-only cap(매치 강도 기준), global cap, 수용 임계값, 모호성 마진, 충돌 감점, 등록 추적 깊이 — 수치는 오직 여기에만

A2 · 출처 그룹화

사이트 내 MAX·사이트 간 noisy-OR인 이유와, 그것이 막는 이중 계산 오류

A3 · 지정 vs 집합 AST

지정 초기화를 구조적으로 구분되게 만드는 lowering 차이(멤버 대입 래핑)